



Cascading Style Sheets (CSS)

Objectives

- To control the appearance of a website by creating style sheets
- To use a style sheet to give all the pages of a website the same look and feel
- To use the class attribute to apply styles
- To specify the precise font, size, color and other properties of displayed text



What is CSS?

- **CSS** stands for **C**ascading **S**tyle **S**heets
- Cascading Style Sheets is the W3C standard for defining the presentation (feel and look) of web pages.
- CSS describes **how HTML elements are to be displayed on screen, paper, or in other media**
- **CSS saves a lot of work.**
- It can control the layout of multiple web pages all at once

Cascading Style Sheets (CSS)

- CSS is for **styling the web pages** and the HTML can be reserved to handle the business of **defining document structure and meaning**.
- CSS allows document authors to specify the presentation of elements on a web page(e.g., fonts type, spacing, colors,position) separately from structure of the document (headers, body, links).
- This separation of structure from presentation simplifies maintaining and modifying a web page



How Style Sheets Work

It's as easy as 1-2-3!

- **STEP1:** Start with a document that has been marked up in HTML.
- **STEP2:** Write style rules for how you'd like certain elements to look.
- **STEP3:** Attach the style rules to the document.
- When the browser displays the document, it follows your rules for rendering elements.

Anatomy of a Style

- CSS is a separate language with its own syntax.
- A style sheet is made up of one or more style instructions (called **rules** or **rule sets**)
- These rules describe how an element or group of elements should be displayed.
- Each rule *selects* an element and *declares* how it should look.

- Refer to this example

```
h1 { color: green; }
```

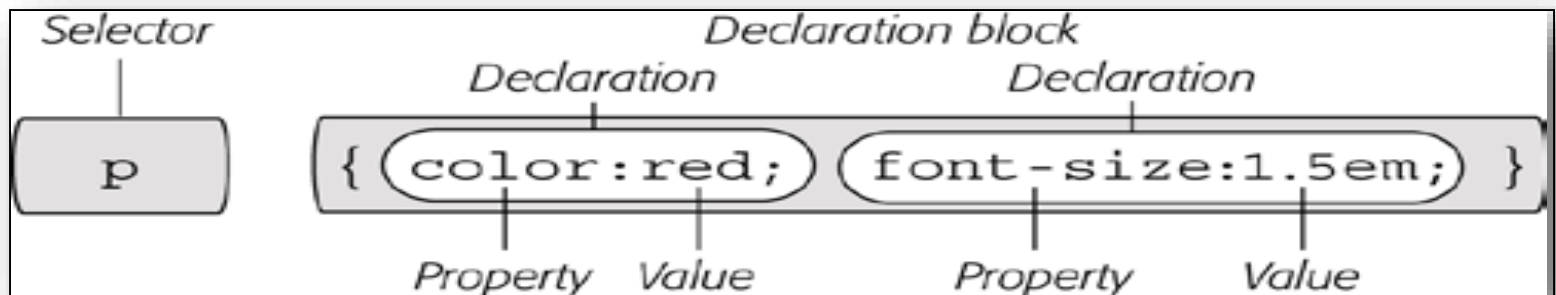
The 1st makes all the **h1** elements in the document green

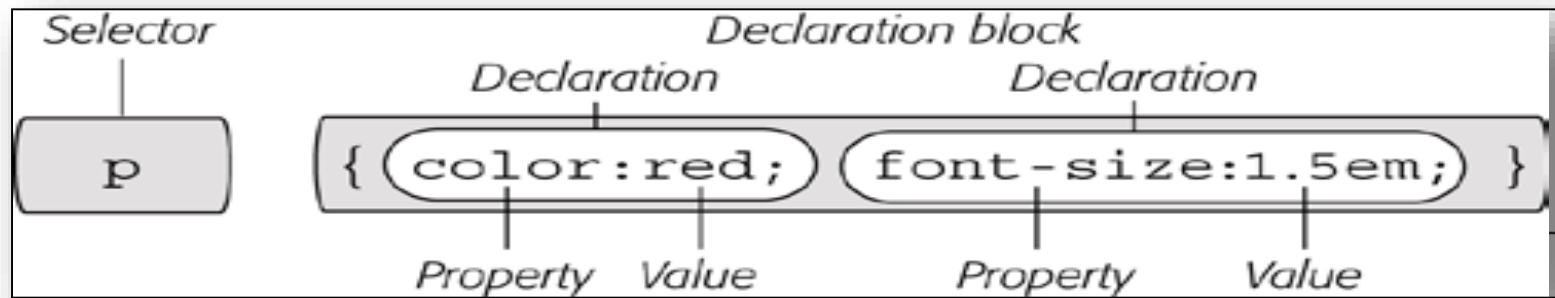
The 2nd specifies that the p element should be in a small, sans-serif font.

```
p { font-size: small; font-family: sans-serif; }
```

Anatomy of a Style

- The CSS is made up of two main sections the **selector** and **declaration block**.





- The **selector** points to the HTML element you want to style.
- The **declaration block** contains one or more declarations separated by semicolons.
- Each **declaration** includes a CSS property name and a value, separated by a colon.
- A **CSS declaration** always ends with a semicolon, and declaration blocks are surrounded by curly braces.

Anatomy of a style: element selector

- CSS selectors are used to "find" (or select) HTML elements based on their element name, id, class, attribute, and more.
- The selector tells a web browser which element or elements on a page to style like a headline, paragraph, image, or link.
- For example:

```
p {  
    color: red;  
    text-align: center; text-align:center;}
```

the p selector refers to the <p> tag.



Anatomy of a style: The id Selector

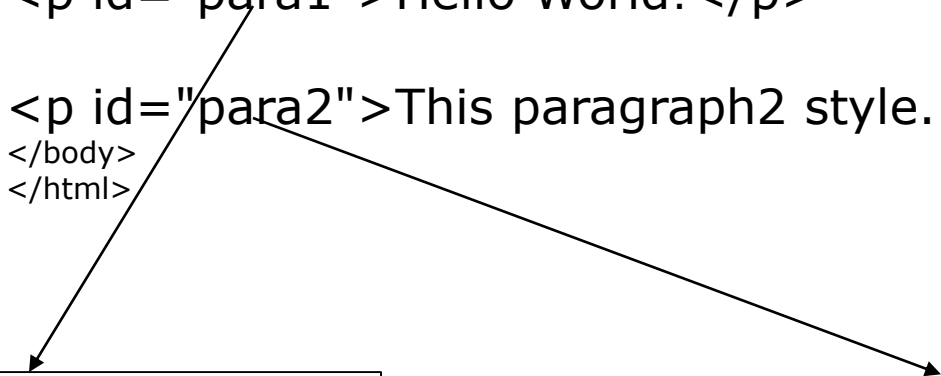
- The id selector uses the id attribute of an HTML element to select a specific element.
- The id of an element should be unique within a page, so the id selector is used to select one unique element!
- To select an element with a specific id, write a hash (#) character, followed by the id name of the element.

Anatomy of a style: The id Selector

- The style rule below will be applied to the HTML element with id="para2":

```
<html>
<head>
</head>
<body>
<p id="para1">Hello World!</p>

<p id="para2">This paragraph2 style.</p>
</body>
</html>
```



```
#para1{
  text-align: left;
  color: blue;}
```

```
#para2{
  text-align: center;
  color: red;}
```



Anatomy of a style: The class Selector

- The class selector selects elements with a specific class attribute.
- To select elements with a specific class, write a period (.) character, followed by the name of the class.

Anatomy of a style: The class Selector

- In the example below, all HTML elements with `class="center"` will be red and center-aligned:

```
<!DOCTYPE html>  
<html>  
<head>
```

```
<style>
```

```
  .center {  
    text-align: center;  
    color: red; }
```

```
  #para1 {  
    text-align: center;  
    color: red;}
```

```
</style>
```

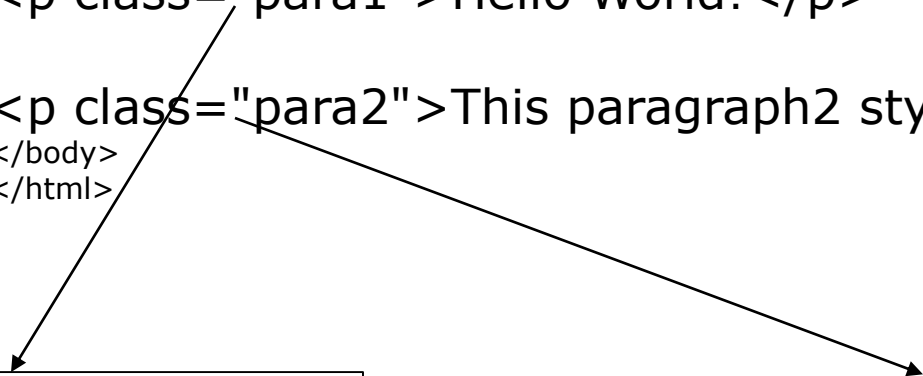
```
</head>  
<body>
```

Anatomy of a style: The class Selector

- The style rule below will be applied to the HTML element with id="para2":

```
<html>
<head>
</head>
<body>
<p class="para1">Hello World!</p>

<p class="para2">This paragraph2 style.</p>
</body>
</html>
```



```
.para1{
  text-align: left;
  color: blue;}
```

```
.para2{
  text-align: center;
  color: red;}
```



Anatomy of a style: The class Selector

- HTML elements can also refer to more than one class.
- In the example below, the `<p>` element will be styled according to `class="center"` and to `class="large"`:

```
<p class="center large">This paragraph refers to two classes.</p>
```

Anatomy of a style: The class Selector

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.center {
```

```
  text-align: center;
```

```
  color: red;
```

```
}
```

```
.large {
```

```
  font-size: 300%;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1 class="center">This heading will not be affected</h1>
```

```
<p class="center">This paragraph will be red and center-aligned.</p>
```

```
<p class="center">This paragraph will be red, center-aligned, and  
font-size.</p>
```

```
</body>
```

```
</html>
```

Anatomy of a style: Grouping Selectors

- Consider the HTML elements with the same style definitions, like this:

```
h1 {  
  text-align: center;  
  color: red;  
}
```

```
h2 {  
  text-align: center;  
  color: red;  
}
```

```
p {  
  text-align: center;  
  color: red;  
}
```




Anatomy of a style: Grouping Selectors

- It will be better to group the selectors, to minimize the code.
- To group selectors, separate each selector with a comma.
- In the example below we have grouped the selectors from the code above:

```
h1, h2, p {  
    text-align: center;  
    color: red;  
}
```



CSS Comments

- Comments are used to explain the code, and is helpful when editing the source code at a later date.
- Comments are ignored by browsers.
- A CSS comment starts with `/*` and ends with `*/`.
- Comments can also span multiple lines

CSS Comments

```
p {  
    /*color: red;  
    This is a single-line comment */  
    text-align: center;  
}
```

```
/* This is  
a multi-line  
comment */
```



Attaching the styles to the document

There are three ways that CSS can be applied to an HTML document.

- Inline styles
- Embedded/Internal styles
- External styles



An external styles

- An external style sheet is a separate, text-only document that contains a number of style rules.
- It must be named with the `.css` suffix.
- The `.css` document is then linked to or imported into one or more HTML documents.
- In this way, all the files in a website may share the same style sheet.
- This is the most powerful and preferred method for attaching style sheets to content.

Linking an external CSS using HTML

- Once you create an external style sheet, you must link it to the web page you wish to format.
- You can attach a style sheet to a web page using HTML's <link> tag as follows:

```
<link rel="stylesheet" type="text/css" href =  
"css/MyStyles.css">
```

Linking an external CSS using HTML

From:

```
<link rel="stylesheet" type="text/css" href =  
"css/MyStyles.css">
```

- **rel="stylesheet"** indicates the type of link—in this case, a link to a style sheet.
- The **type** attribute specifies the document's type for this case is **text/css**
- **href** points to the location of the external CSS file on the site, for this case the css file is in CSS folder.

Linking an external CSS using HTML

- Once you create an external style sheet, you must link it to the web page you wish to format.
- You can attach a style sheet to a web page using HTML's <link> tag as follows:

```
<link rel="stylesheet" type="text/css" href =  
"css/MyStyles.css">
```

- The <link> element goes inside the <head> section:

```
<head>  
<link rel="stylesheet" type="text/css" href="  
mystyle.css">  
</head>
```


Linking an external CSS using HTML

- The <link> element goes inside the <head> section:

```
<head>  
<link rel="stylesheet" type="text/css"  
      href="mystyle.css">  
</head>
```

Linking an external CSS using HTML

- An external style sheet can be written in any text editor. The file should not contain any html tags.
- The style sheet file must be saved with a .css extension.
- Here is how the "MyStyles.css" looks:

```
body {  
    background-color: lightblue;  
}  
  
h1 {  
    color: navy; margin-left: 20px;  
}
```



External styles: Demonstration

- Refer to external folder

Linking an external CSS using CSS

- CSS includes a built-in way to add external style sheets—the **@import directive**.
- You add the directive inside of an HTML <style> tag, like so:

```
<style type="text/css" >  
    @import url(style1.css);  
    @import url(style2.css);  
    @import url(style3.css);  
</style>
```

- As with the <link> tag, you can include multiple external style sheets by using more than one @import

Embedded styles

- It is placed in a document using the **style** element, and its rules apply only to that document.
- The **style** element must be placed in the **head** of the document.
- Refer the example below

... <head>

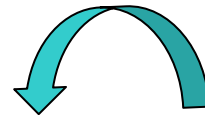
<title>Required document title here</title>

<style >

/* CSS rules go here */

</style>

</head> ...



Comments in Style Sheets

Embedded style: Example 01

<head>

 <title>Style Sheets</title>

 <!-- this begins the style sheet section -->

 <style type = "text/css">

 b{ font-weight: bold; color: black }

 h1{ font-family: tahoma, helvetica, sans-serif }

 p{ font-size: 12pt;font-family: arial, sans-serif }

 .special { color: #6666ff }

 </style>

 <!-- end of style sheet section -->

</head>



Embedded style: Example01-Explained

- The **style** element defines the embedded style sheet.
- The **type** attribute describes a file's content
- In this case of CSS, type = "text/css"
- Inside body of the **style sheet** declares the CSS rules for the style sheet.

Embedded style: Example01-Explained

- Applying the .special style selector

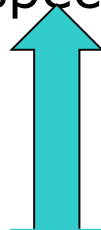
-
-
-

<!-- this class attribute applies the .special style -->

<h1 class = "special">Deitel & Associates, Inc.</h1>

<p class = "special" >The use of special selector </p>

-
-
-



The selector class special is called here

Inline Styles

- With inline styles properties and values are applied to a single element using the **style** attribute in the element itself, as shown here:

<h1 style="color: red">Introduction</h1>

- To add multiple properties, just separate them with semicolons, like this:

<h1 style="color: red; margin-top: 2em">Introduction</h1>.

- Inline styles apply only to the particular element in which they appear.
- Inline style is considered problematic because it does not truly separate presentation from structure.



Inline

- [Exercise 2](#) gives you an opportunity to write an inline style and see how it works.
- We won't be working with inline styles after this point for the reasons listed earlier, so here's your chance.



Conflicting styles: the cascade

- CSS allows you to apply **several style sheets** to **the same document**, which means there are **bound to be conflicts**.
- For example, what should the browser do if a document's **linked style sheet** says that **h1** elements **should be red**, but its **embedded style sheet** has a rule that makes **h1 purple**, while inline says it should be rendered with colour **yellow**?



Conflicting styles: the cascade

- If there is no any style information to a web page, it will be rendered according to the browser's internal style sheet.
- Individual users can apply their own styles as well (the **user style sheet**), which overrides the default styles in their browser.
- Now, what actually is the determinant for a style to win the battle?



Conflicting styles: the cascade

- The **style sheet source** is one hierarchy that determines which style wins.
- Generally speaking, the **closer** the style sheet is to the content, the **more weight** it is given.
- **Embedded** style sheets that appear right in the document in the **style** element have more weight than **external** style sheets.
- **Inline styles** have more weight than **embedded** style sheets because you can't get any closer to the content than a style right in the element's opening tag.



Preventing overwriting the CSS rule

- If you want a rule not to be overridden by a subsequent conflicting rule, include the **!important** indicator just after the property value and before the semicolon for that rule.
- For example, to make paragraph text blue always, use the following rule:

```
p {color: blue !important;}.
```

Preventing overwriting the CSS rule

- For example, to make paragraph text blue always, use the following rule:
`p {color: blue !important;}`.
- Even if the browser encounters an inline style later in the document like this one:
`<p style="color: red">`
- Such paragraph will still be blue because the rule with the **!important** indicator cannot be overridden by other.

Overwriting the !important rule

- The only way an **!important** rule may be overridden is by a conflicting rule in a reader (user) style sheet that has also been marked **!important**.
- This is to ensure that special reader requirements, such as large type for the visually impaired, are never overridden.
- Basing on previous example if the reader's style sheet includes this rule:

`p {color: black;} the text would still be blue`
- But with `p {color: black !important;}` the paragraphs will be black and cannot be overridden by any author-provided style.

Style Sheet Hierarchy

Style information can come from various sources, listed here from general to specific.

Items lower in the list will override items above them:

1. Browser default settings
2. User style settings (set in a browser as a “reader style sheet”)
3. Linked external style sheet (added with the **link** element)
4. Imported style sheets (added with the **@import** function)
5. Embedded style sheets (added with the **style** element)
6. Inline style information (added with the **style** attribute in an opening tag)
7. Any style rule marked **!important** by the author
8. Any style rule marked **!important** by the reader (user)

Overwritten by

Overwrites

Rule order

- Finally, if there are conflicts within style rules of identical weight, whichever one comes last in the list “wins.” Take these three rules, for example:

`<style>`

`p { color: red; }`

`p { color: blue; }`

`p { color: green; }`

`</style>`

- In this scenario, paragraph text will be green because the last rule in the style sheet—that is, the one closest to the content in the document—overrides the earlier ones.

Rule order

- The same thing happens when conflicting styles occur within a single declaration stack:

`<style>`

```
p { color: red;  
    color: blue;  
    color: green; }
```

`</style>`

- The resulting color will be green because the last declaration overrides the previous two.



TUTORIALS

- **Creating an Inline Style**
- **Creating an embedded/internal Style**
- **Creating an External Style**



Tutorial: External CSS

- This guide explains how to use the Google Fonts API to add fonts to your web pages.
- All you have to do is add a special stylesheet link to your HTML document, then refer to the font in a CSS style.

https://developers.google.com/fonts/docs/getting_started

CSS Applied

Adding Text Shadow

- The text-shadow property requires four pieces of information:
 - The horizontal offset
 - The vertical offset
 - The blurriness
 - The colour

`text-shadow: (-4px) (4px) 3px #999999;`

Text Shadow

`text-shadow: -4px 4px 3px #999999, 1px -1px 2px #000;`

Text Shadow

Adding Text Shadow

- The text-shadow property requires four pieces of information:
 - The horizontal offset
 - How far to the left or right of the text the shadow should appear,
 - - values means left,
 - + values means right

text-shadow: -4px 4px 3px #999999;

Text Shadow

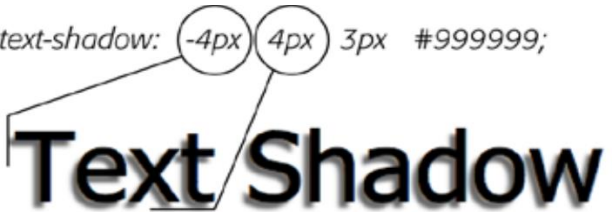
Text Shadow

text-shadow: -4px 4px 3px #999999, 1px -1px 2px #000;

Adding Text Shadow

- The text-shadow property requires four pieces of information:
 - The horizontal offset
 - How far to the left or right of the text the shadow should appear,
 - - values means left,
 - + values means right

`text-shadow: -4px 4px 3px #999999;`



Text Shadow

`text-shadow: -4px 4px 3px #999999, 1px -1px 2px #000;`



Text Shadow

Qn: What does it mean by the term horizontal offset of the text-shadow?

Qn: What does it mean if the horizontal offset value of the text-shadow is negative?

Adding Text Shadow

- The vertical offset
 - How far to the top or below of the text the shadow should appear,
 - - values means top,
 - + values means below

`text-shadow: (-4px) (4px) 3px #999999;`

Text Shadow

Text Shadow

`text-shadow: -4px 4px 3px #999999, 1px -1px 2px #000;`

Qn: What does it mean by the term vertical offset of the text-shadow?

Qn: What does it mean if the vertical offset value of the text-shadow is positive?

Adding Text Shadow

- the blurriness of the shadow, and
- the color of the drop shadow.

`text-shadow: -4px 4px 3px #999999;`

Text Shadow

Text Shadow

`text-shadow: -4px 4px 3px #999999, 1px -1px 2px #000;`

Adding Text Shadow

- For example, here's the text-shadow property that creates the effect at the top of Figure:
 - text-shadow: -4px 4px 3px #999999; means the shadow is:
 - To the left
 - Below
 - Blurred by 3px
 - with #999999 colour

text-shadow: (-4px) (4px) 3px #999999;

Text Shadow

Text Shadow

text-shadow: -4px 4px 3px #999999, 1px -1px 2px #000;

QN: What does this property mean text-shadow: +4px -4px 3px #999999?

Adding Text Shadow

- If we change the values from -6px 8px 3px #999999; to 6px -8px 6px #999999 for h1:

- #sha1{text-shadow: **-4px 4px 3px** #999999; }

IS088

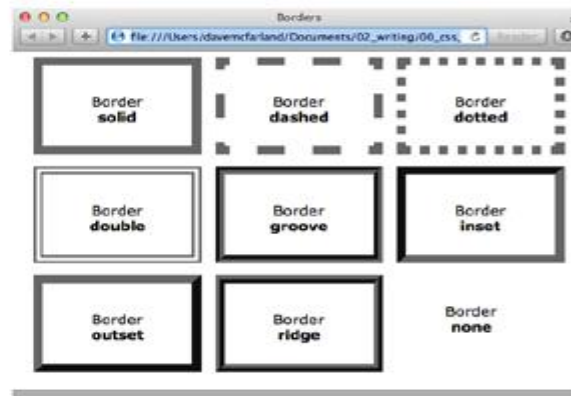
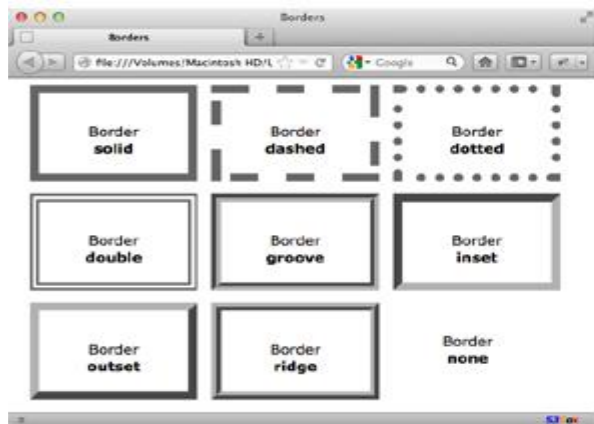
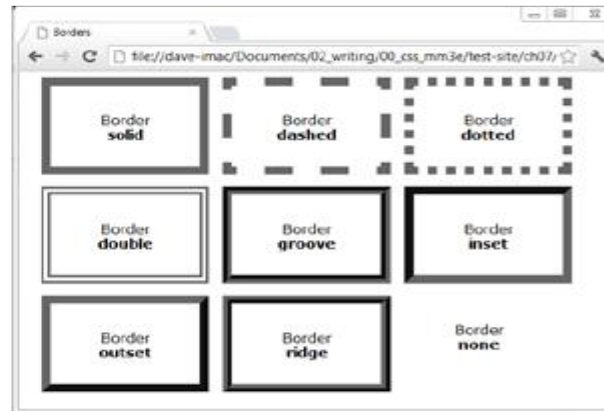
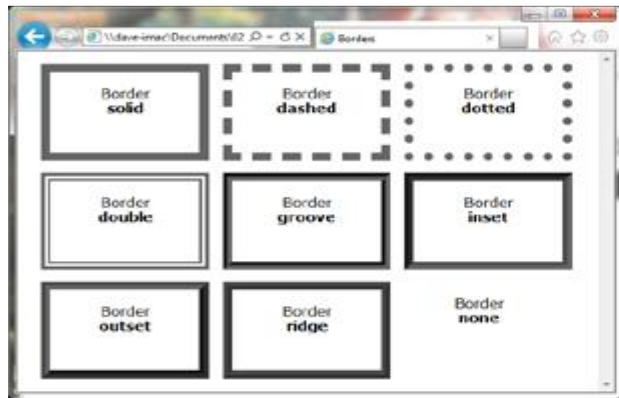
- #sha2{text-shadow: **4px -4px 6px** #999999;}

IS088

Adding Borders

- A border is simply a line that runs around an element.
- You can add a border to all four edges using CSS
- You control three different properties of each **border: width, style, and color.**
 - Width is for the thickness of the line
 - The color can be any of the CSS color
 - CSS offers these styles: solid, dotted, dashed, double, groove, ridge, inset, outset, none, and hidden

Adding Borders



Declaring the Border Property

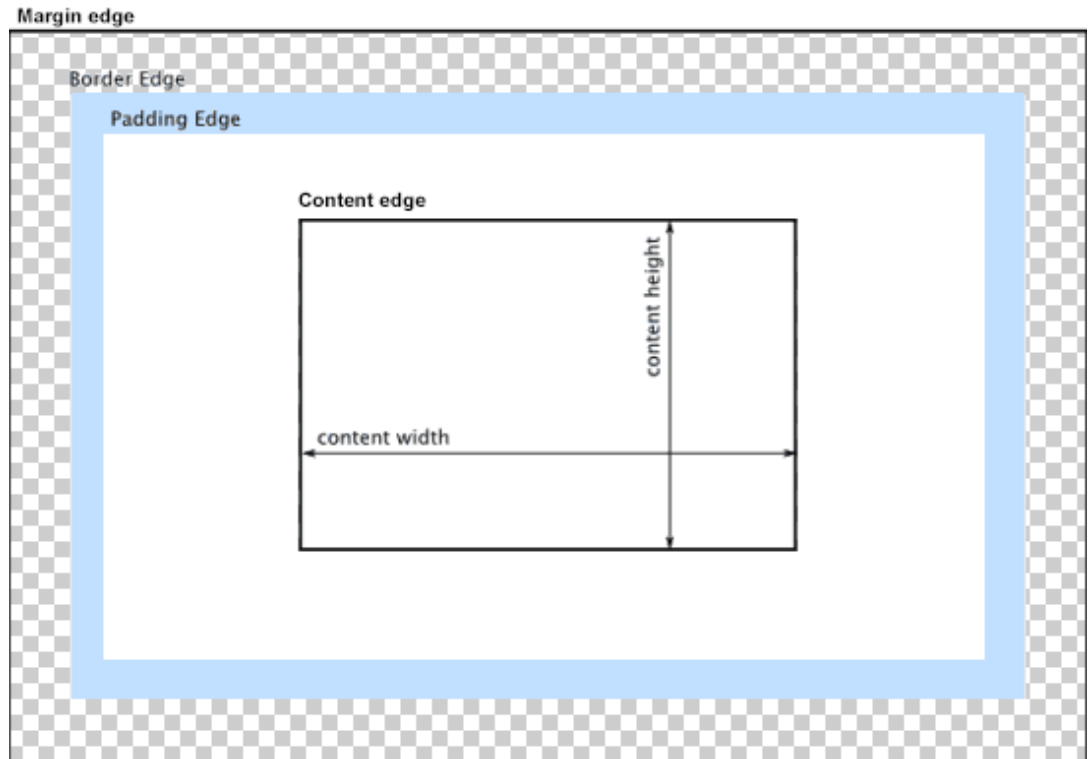
```
P{  
border: 4px solid #FF0000;  
}
```


Formatting Individual Borders

- You can control each border individually using the appropriate property:
 - Border-top, border-bottom, border-left, or border-right
- For example:
 - `border-top: 2px solid black;`
 - `border-left: 2px solid black;`
 - `border-right: 2px solid black;`
 - `border-bottom: 4px dashed #333;`
- What if we write:
 - `border: 2px solid black;`
 - `border-bottom: 4px dashed #333;`
- It uses the CSS cascading rule discussed earlier

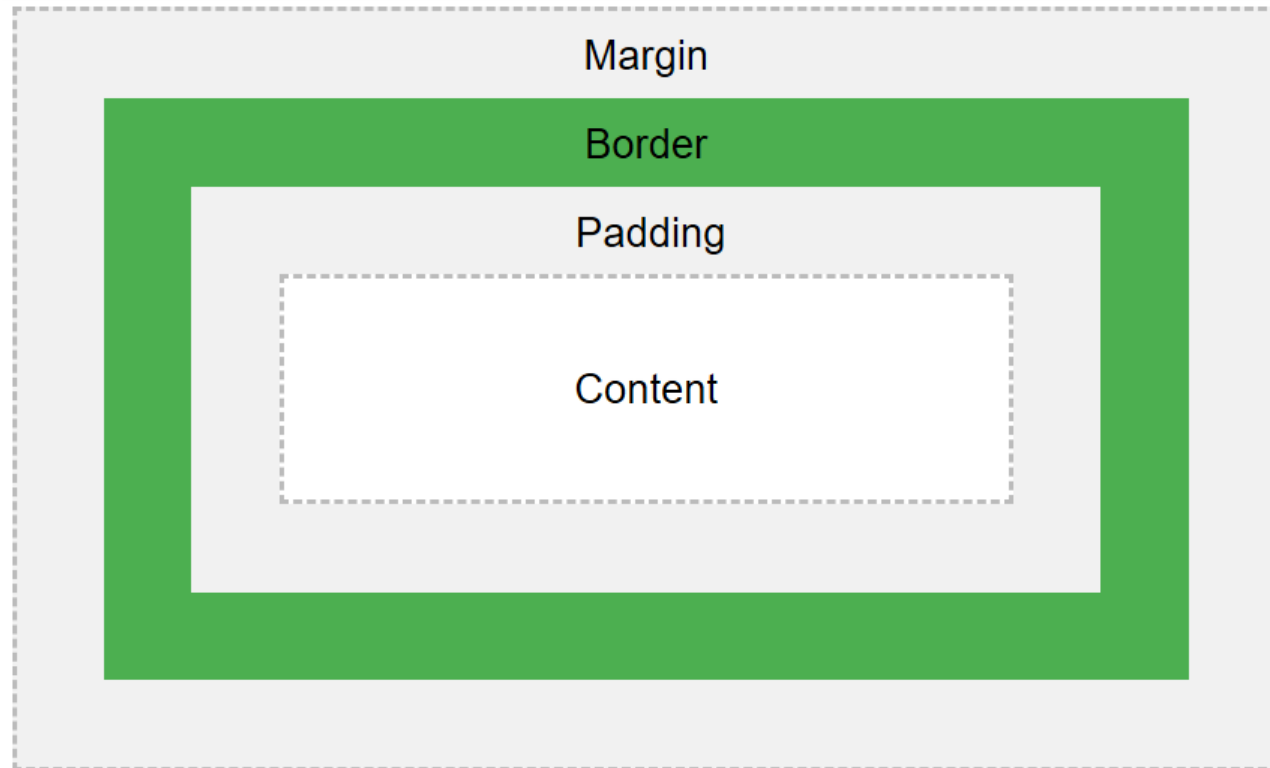
CSS BOX Model

- All HTML elements can be considered as boxes.
- In CSS, the term "box model" is used when talking about design and layout.
- The CSS box model is essentially a box that wraps around every HTML element.
- It consists of: **margins**, **borders**, **padding**, and the actual **content**.
- This image illustrates the box model:

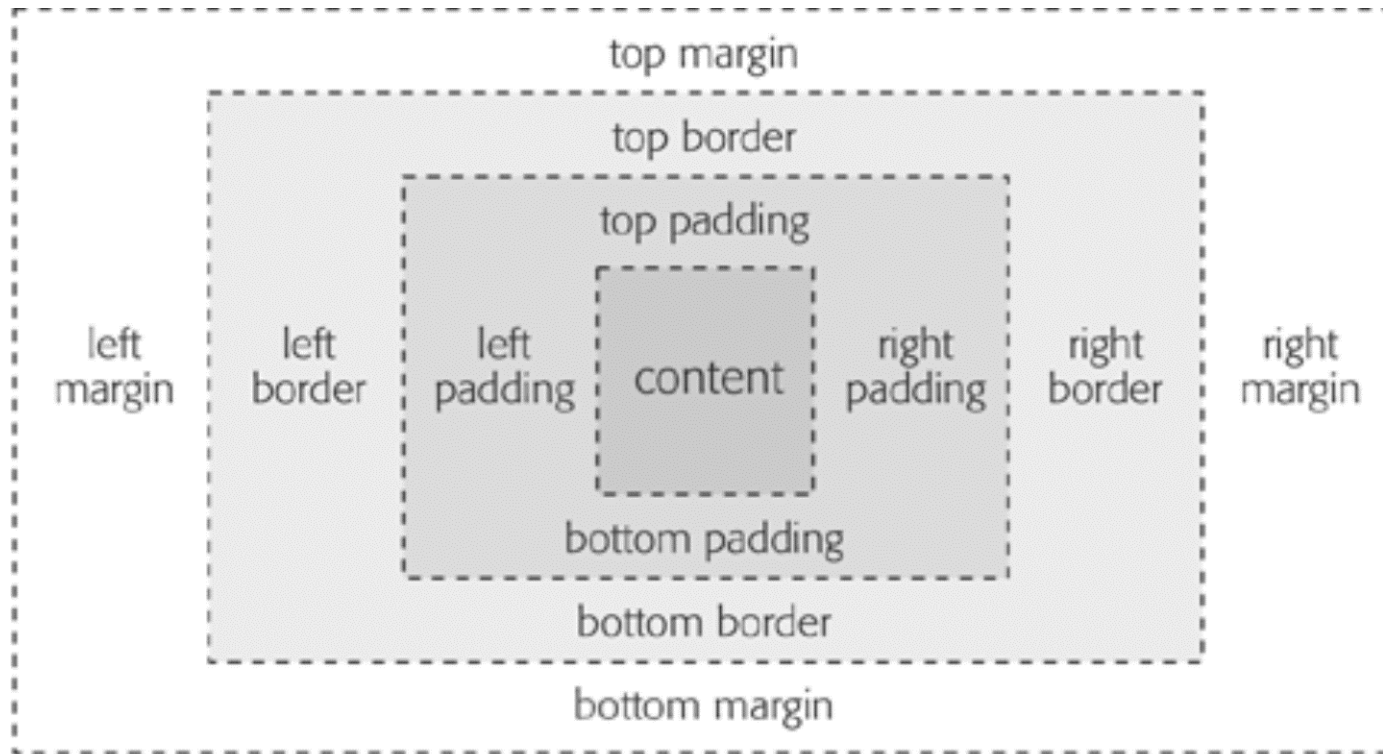


CSS BOX Model

- All HTML elements can be considered as boxes.
- In CSS, the term "box model" is used when talking about design and layout.
- The CSS box model is essentially a box that wraps around every HTML element.
- It consists of: **margins**, **borders**, **padding**, and the actual **content**.
- This image illustrates the box model:

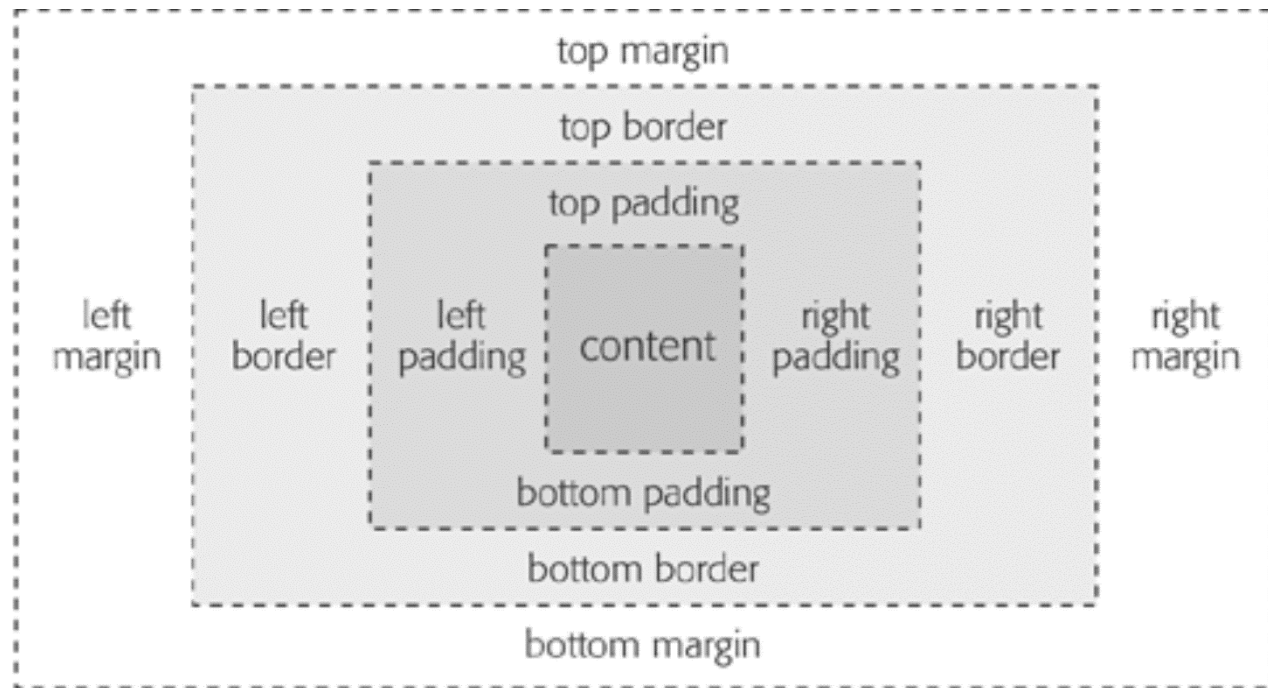


BOX Model: Margins, Padding, and Borders



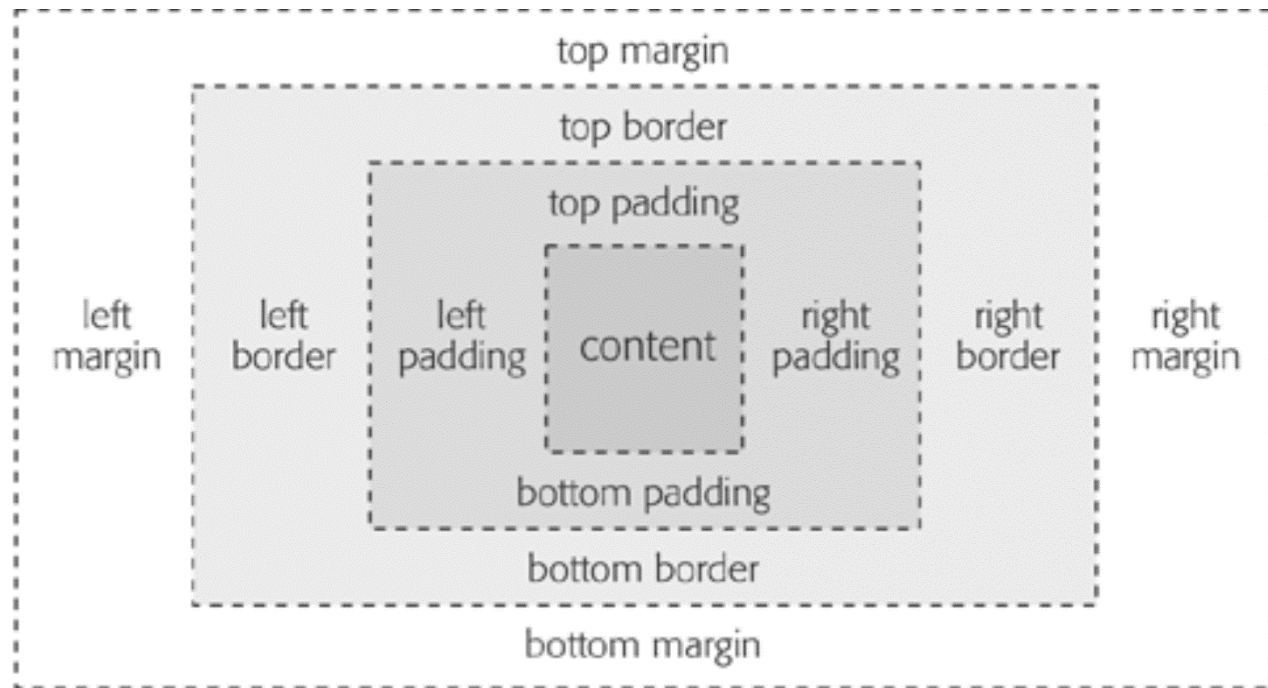
Understanding the box model

- **padding** is the space between the content and the content's border.
- Padding is what separates a photo from the border that frames the photo.



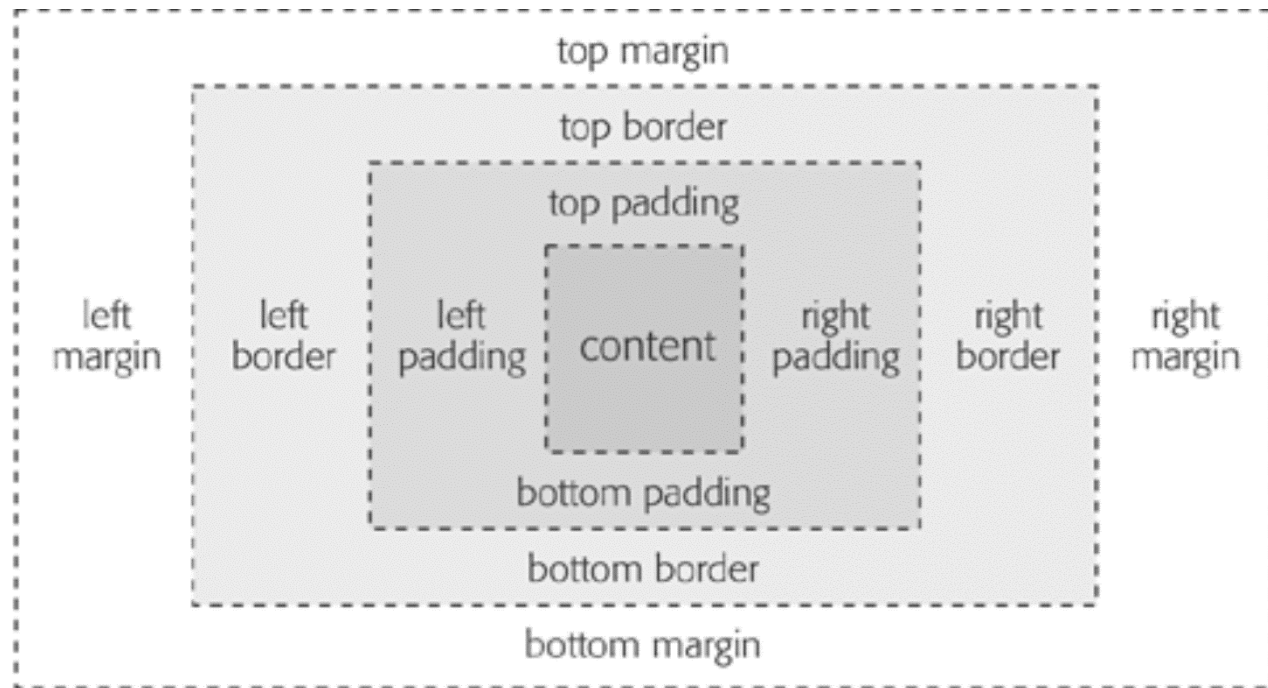
Understanding the box model

- **border** is the line that's drawn around each edge of the box.
- You can have a border around all four sides, on just a single side, or any combination of sides.

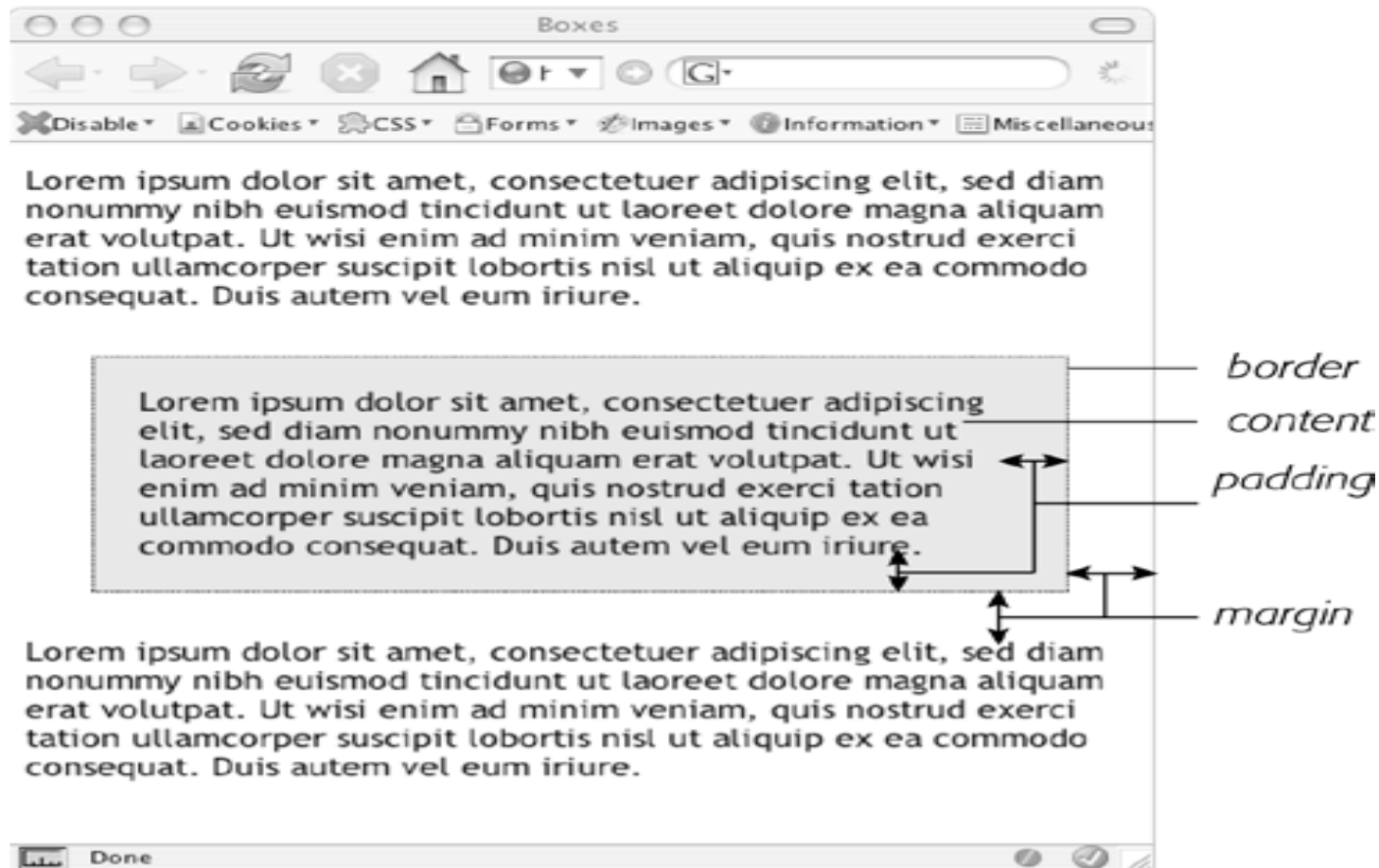


Understanding the box model

- **margin** is what separates one tag from another.
- The space that commonly appears between the tops and bottoms of paragraphs of text on a web page, for example, is the margin.

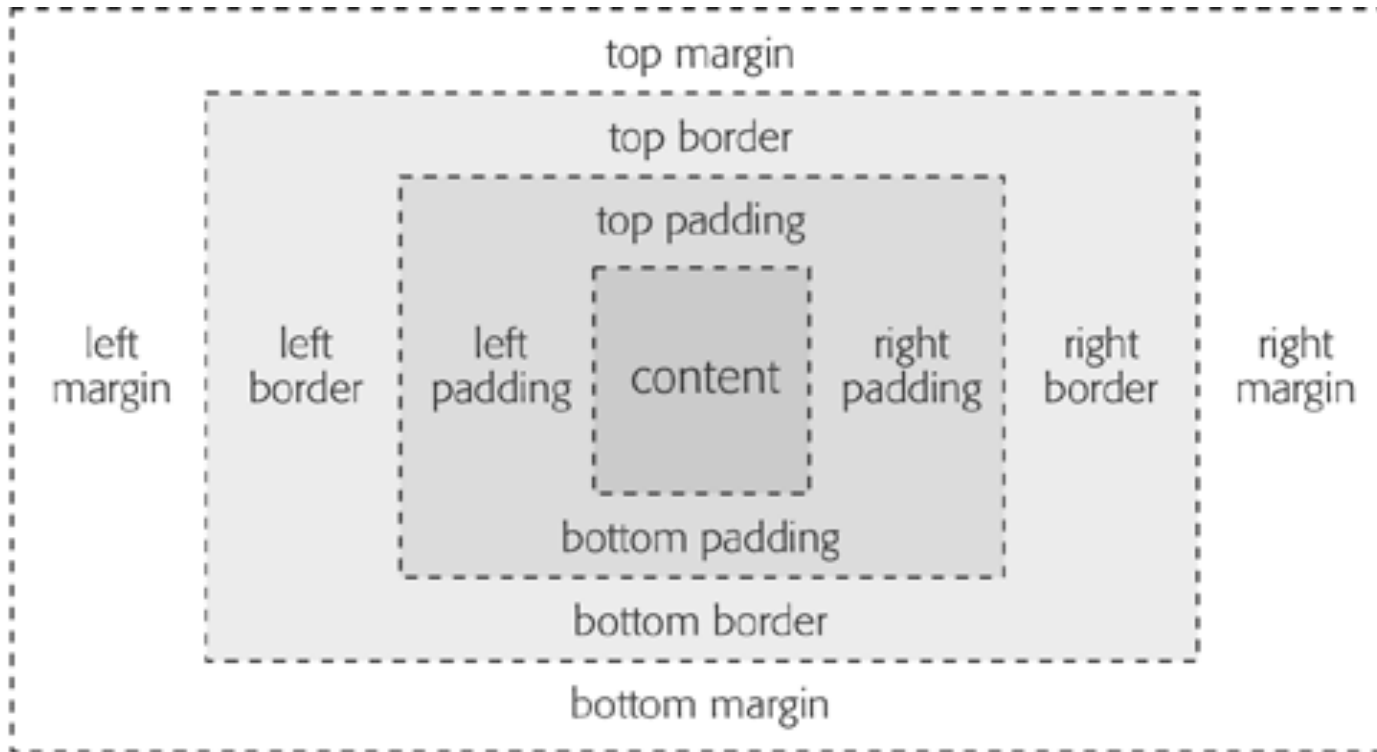


Understanding the box model



[Check Demo:](#)
[Demo1.html](#)

Understanding the box model



For a given tag, you can use any or all of these properties in combination. You can set just a margin for a tag or add a border, margins, and padding

Controlling the Margin

- Four properties control margin edges: margin-top, margin-right, margin-bottom, and margin-left.
- All the margin properties can have the following values:
 - auto - the browser calculates the margin
 - length - specifies a margin in px, pt, cm, etc.
 - % - specifies a margin in % of the width of the containing element
 - inherit - specifies that the margin should be inherited from the parent element
- For example

```
P{  
margin-right: 20px;  
margin-left: 10%;  
}
```

Controlling the Padding

- Similarly, four properties control padding: padding-top, padding-right, padding-bottom, and padding-left.
- For example

```
h1{  
padding-right: 20px;  
padding-left: 10%;  
}
```

Margin and Padding Shorthand

- Typing out all four properties (margin-top, margin-right, margin-bottom, and margin-left) for each style is tedious.
- You can use the shortcut properties named margin and padding to set all four properties quickly:

```
P{  
    margin: 0 10px 10px 20px;  
    padding: 10px 5px 5px 10px; }
```

- The order in which you specify the four values is important. It must be top, right, bottom, and left.
- Note: `p{margin: 10px 10px 10px 10px;}` is same as `P{margin:10px;}`

Building Float-Based Layouts

- Float-based layouts take advantage of the float property to position elements side by side and create columns on a web page
- The float property moves a page element to one side of the page (or other containing block).
- The float property accepts one of three different values:
 - float:left;
 - float:right; and
 - float:none;

Building Float-Based Layouts

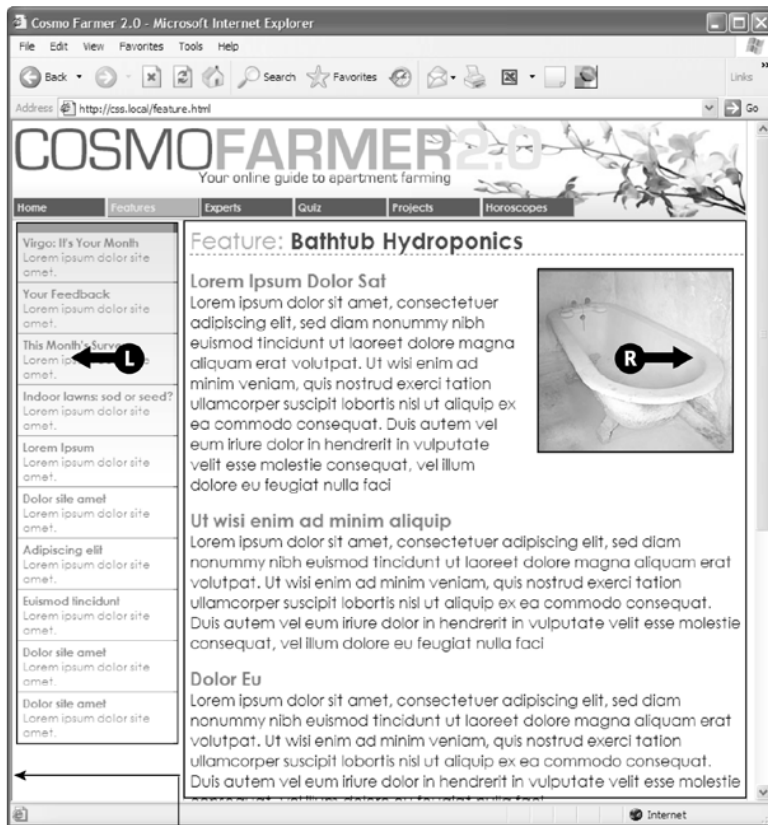
- To move an image to the right side of the page, you could create this class style and apply it to the tag:

```
.floatRight {  
    float: right;  
}
```

- The same property applied to a <div> tag full of content can also create a sidebar:

```
.sidebar {  
    float: left;  
    width: 170px;  
}
```

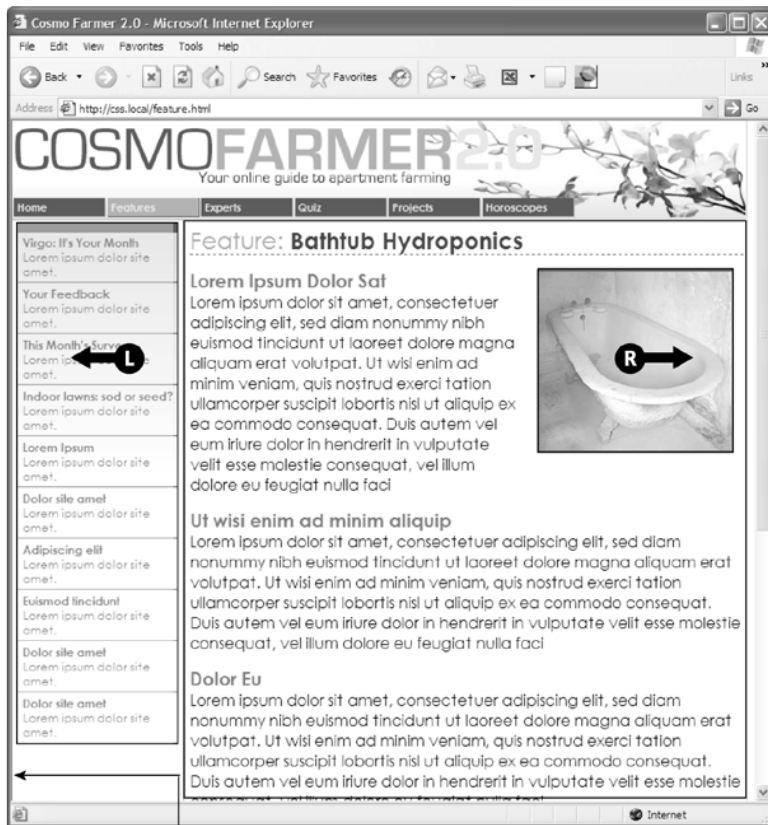
Building Float-Based Layouts



left margin

- You can use the float property to lay out a web page with multiple columns.
- On this page, a block of content is floated to the left edge.
- The sidebar has a set width, but the main content doesn't, which makes this design a liquid layout.

Building Float-Based Layouts



- The main section of the page simply expands to fill the width of the browser window.
- In the upper right, the bathtub photo is floated to the right.

HTML file

<!-- write something resembling to this , not necessarily similar contents -->

<html>

<head>

<title> The three columns layouts </title>

</head>

<body>

I am the header

I am the Left column for the navigation

I am the Main column

I am the right column for the upcoming News

I am the footer

</body>

</html>

HTML file: <!-- Apply the DIV as shown -->

<body>

<div> I am the header</div>

<div> I am the Left column for the navigation</div>

<div> I am the Main column</div>

<div> I am the right column for the upcoming News</div>

<div> I am the footer</div>

</body>

Class *sidebar1*:

Create a css rules and save it as my_style.css

```
.sidebar1 {  
    float: left;  
    width: 15%;  
    height:90%;  
    padding: 0 20px 0 10px;  }
```

Class *main*:

```
.main {  
    float: left;  
    width: 60%;  
    height: 90%;  
    padding: 0 20px 0 20px;  
    border-left: dashed 1px blue;  
    border-right: dashed 1px green;}
```

Class *sidebar2*

```
.sidebar2 {  
    float: right;  
    width: 15%;  
    height: 90%;  
    padding: 0 10px 0 20px; }
```

Class 04

```
.footer {  
    clear:both; <!--so that it below ALL-->  
    border-top: dashed 1px green;  
    text-align: center;}
```

Class 05

```
.header {  
    margin-top:2%;  
    height:6%;  
    border-bottom: dashed 1px green;  
    text-align: center;}
```

...applying the CSS rules

<!-- this time we

Apply the CSS rules as shown here

-->

<body>

<div class="header"> I am the header</div>

<div class="sidebar1"> I am the Left column for the navigation</div>

<div class="main"> I am the Main column</div>

<div class="sidebar2"> I am the right column for the upcoming News</div>

<div class="footer"> I am the footer</div>

</body>

Link the CSS to your doc

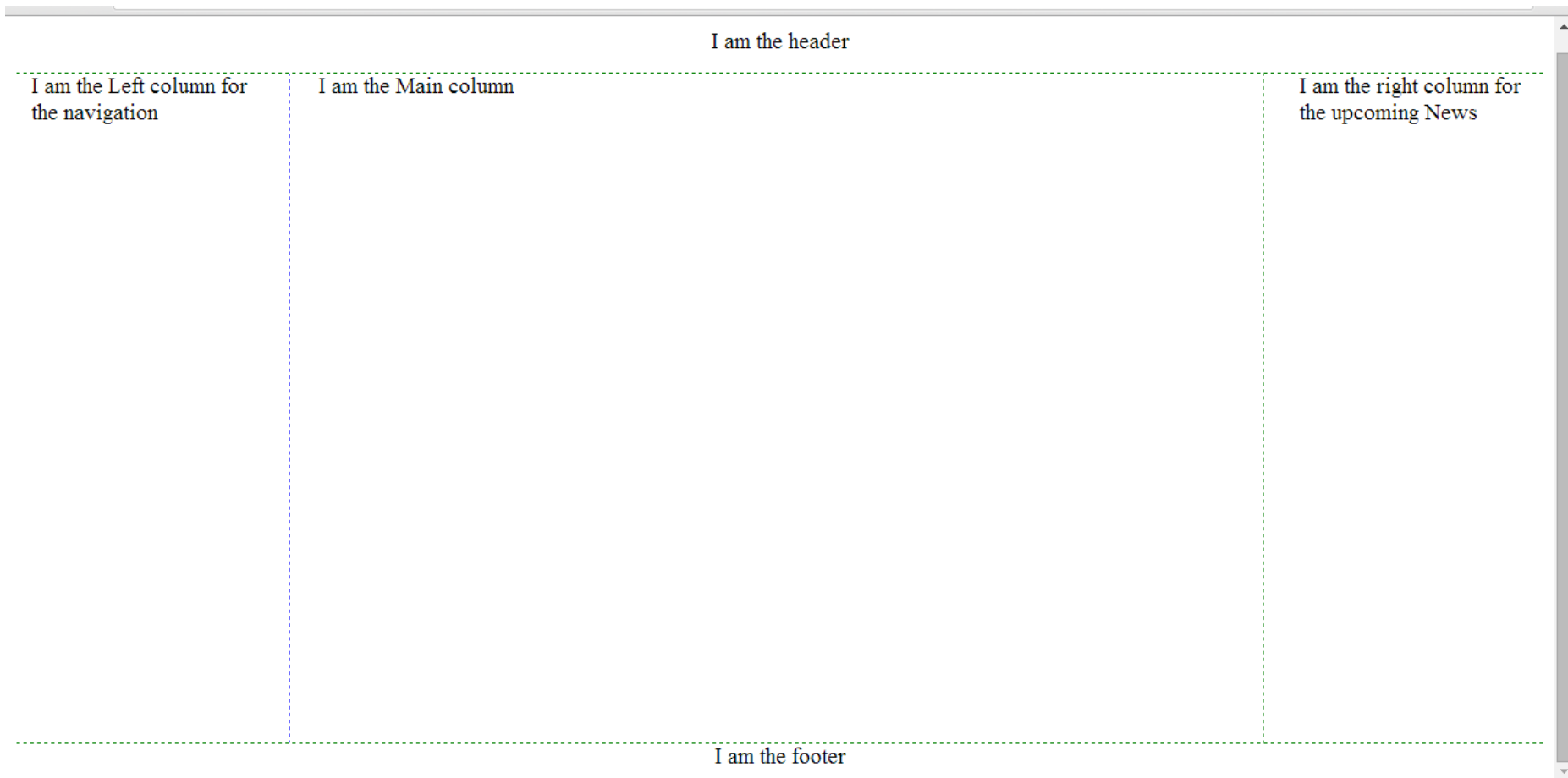
```
<head>
```

```
<link rel="stylesheet" type="text/css" href = "my_style.css">
```

```
</head>
```

...and then preview your page

output

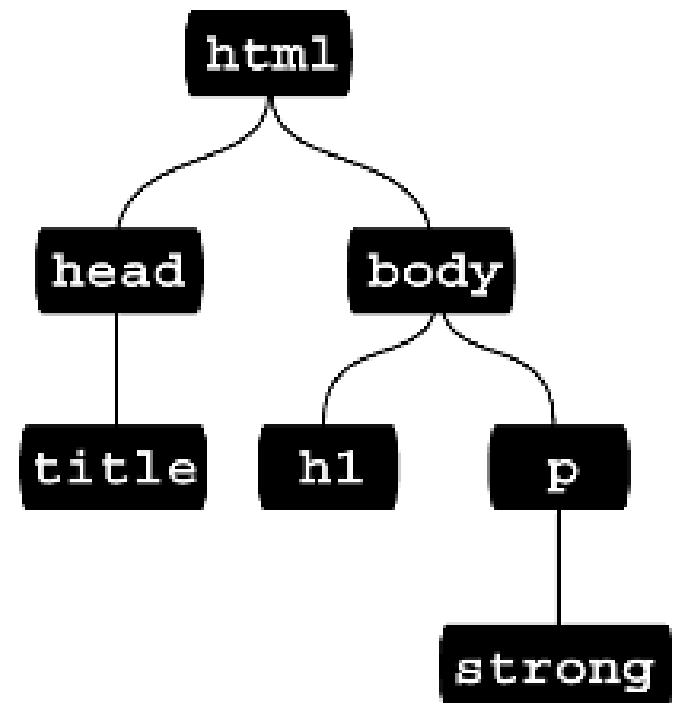


Reading

- **The HTML Document Tree**
 - **Ancestor element**
 - **Descendant element**
 - **Parent and Child elements**
 - **Sibling element**

The HTML family Tree – Nested Tag

```
<html>  
<head>  
<title>A Simple Document</title>  
</head>  
<body>  
<h1>Header</h1>  
<p>A paragraph of  
<strong>important</strong>text.</p>  
</body>  
</html>
```

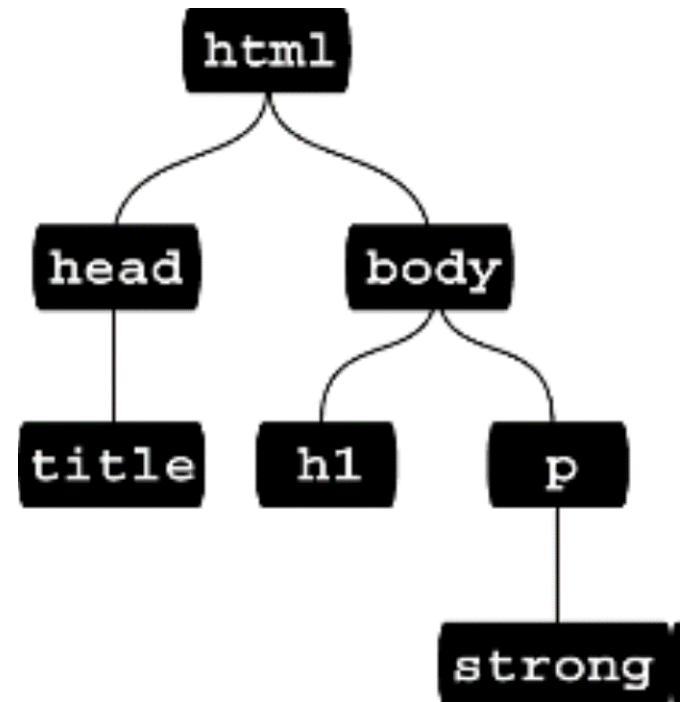


The HTML family Tree - Relationship

- We, human, have grandparents, parents, brothers and sisters, children
- And so are the elements on a page. They as well have more similar relationship.
- The most important relationships are:
 - Ancestor
 - Descendant
 - Parent
 - Child
 - Sibling

The HTML family Tree - Relationship

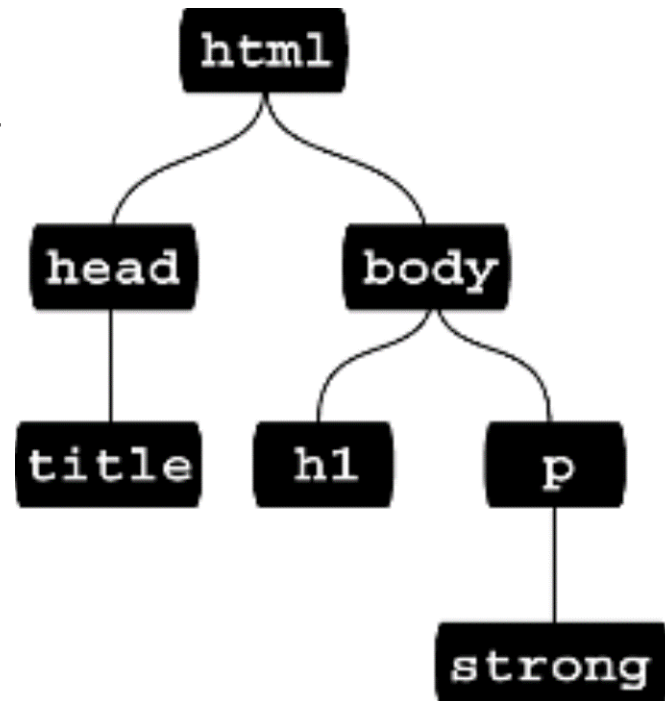
- Ancestor:
 - This is an HTML tag that wraps around another tag.
 - In this figure, the <html> tag is an ancestor of all other tags
 - The <body> is an ancestor for all of the tags inside of it – the <h1>, <p>, and tags



The HTML family Tree - Relationship

- **Descendant:**

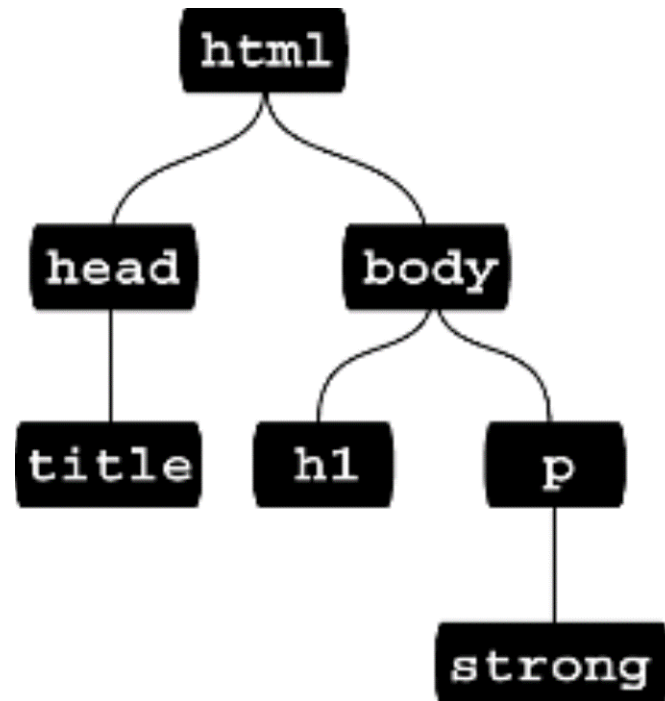
- This is an HTML tag that is inside one or more tags.
- In this figure, the `<body>` tag is a descendant of the `<html>` tag
- The `<p>` tag is a descendant of both the `<body>` and `<html>`



The HTML family Tree - Relationship

- Parent:

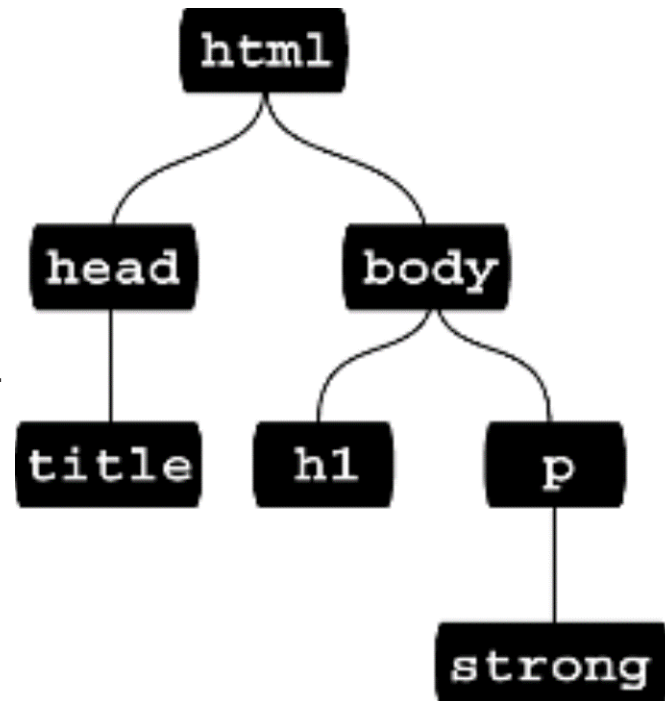
- This refers to the closest ancestor of another tag.
- That's to say; a parent is the first tag directly connected to and above another tag.
- In this figure, the `<html>` tag is a parent of the `<head>` and `<body>` tag.
- The `<head>` tag is not the parent of neither `<p>` nor `<strong>` tags.
- The `<p>` is the parent of the `<strong>` tag.



The HTML family Tree - Relationship

- Child:

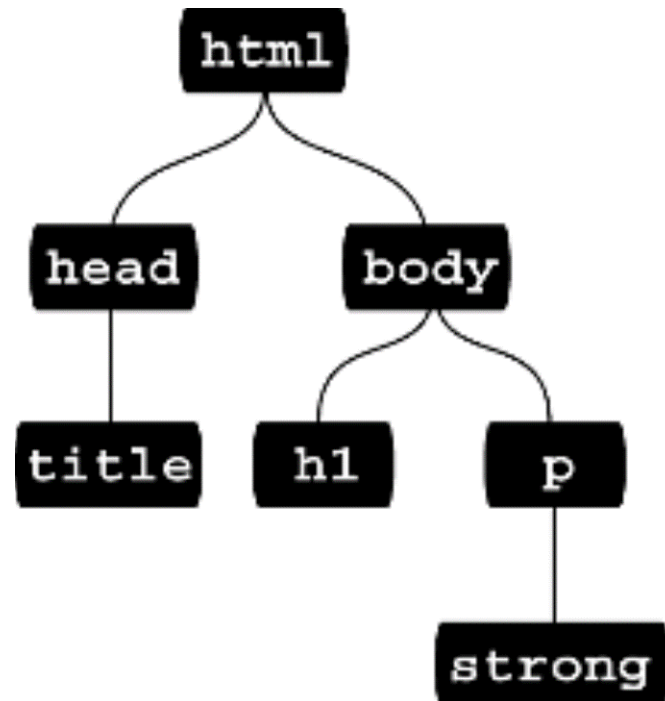
- This refers to the tag that's directly enclosed by another tag.
- In this figure, both the `<h1>` and `<p>` tags are children of the `<body>` tag.
- The `<strong>` tag is directly wrapped inside `<p>` tag, it's a child of the `<p>` tag.



The HTML family Tree - Relationship

- sibling:

- These are tags that are children of the same tag, just like sisters and brothers.
- In this figure, both the <h1> and <p> tags are sibling of the <body> tag.
- They are often next to each other and connected to the same parent
- In this Figure for instance:
- The <head> and <body> tags are siblings



CSS Combinators

- A CSS selector can contain more than one simple selector.
- Between the simple selectors, we can include a combinator.
- A CSS combinator is something that explains the relationship between the selectors.
- There are four different combinators in CSS:
 - descendant selector (**space**)
 - child selector (**>**)
 - adjacent sibling (Next-sibling) selector (**+**)
 - general sibling (subsequent-sibling) selector (**~**)

Descendant Selector

- The descendant selector matches all elements that are descendants of a specified element.
- The combinator for this relationship is the **SPACE**
- Example: To select all <p> elements inside <div> elements we write:

```
div p {  
    background-color: yellow;  
}
```

- See example in next slide

Descendant Selector

```
<!DOCTYPE html>
<html>
<head>
<style>
div p {
  background-color: yellow;
}
</style>
</head>
<body>

<h2>Descendant Selector</h2>

<p>The descendant selector matches all elements that are
descendants of a specified element.</p>

<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section><p>Paragraph 3 in the div.</p></section>
</div>

<p>Paragraph 4. Not in a div.</p>
<p>Paragraph 5. Not in a div.</p>

</body>
</html>
```

Descendant Selector

The descendant selector matches all elements that are descendants of a specified element.

Paragraph 1 in the div.

Paragraph 2 in the div.

Paragraph 3 in the div.

Paragraph 4. Not in a div.

Paragraph 5. Not in a div.

Child Selector

- The child selector selects all elements that are the children of a specified element.
- The combinator for this relationship is the symbol **>**
- Example: To select all `<p>` elements that are children of a `<div>` element we write:

```
div > p {  
    background-color: yellow;  
}
```

- See example in next slide

Child Selector

```
<!DOCTYPE html>
<html>
<head>
<style>
div > p {
  background-color: yellow;
}
</style>
</head>
<body>

<h2>Child Selector</h2>
<p>The child selector (>) selects all elements that are the
children of a specified element.</p>

<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section>
    <!-- not Child but Descendant -->
    <p>Paragraph 3 in the div (inside a section element).</p>
  </section>
  <p>Paragraph 4 in the div.</p>
</div>

<p>Paragraph 5. Not in a div.</p>
<p>Paragraph 6. Not in a div.</p>

</body>
</html>
```

Child Selector

The child selector (>) selects all elements that are the children of a specified element.

Paragraph 1 in the div.

Paragraph 2 in the div.

Paragraph 3 in the div (inside a section element).

Paragraph 4 in the div.

Paragraph 5. Not in a div.

Paragraph 6. Not in a div.

Adjacent sibling selector

- The adjacent sibling selector is used to select an element that is directly after another specific element.
- Sibling elements must have the same parent element, and "adjacent" means "immediately following".
- The combinator for this relationship is the symbol **+**
- Example: To select the first <p> element that are placed immediately after <div> elements we write:

```
div + p {  
    background-color: yellow;  
}
```

- See example in the next slide

Adjacent sibling selector

```
<!DOCTYPE html>
<html>
<head>
<style>
div + p {
  background-color: yellow;
}
</style>
</head>
<body>
<h2>Adjacent Sibling Selector</h2>
<p>The + selector is used to select an element that is
directly after another specific element.</p>
<p>The following example selects the first p element that are
placed immediately after div elements:</p>
<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
</div>
<p>Paragraph 3. After a div.</p>
<p>Paragraph 4. After a div.</p>
<div>
  <p>Paragraph 5 in the div.</p>
  <p>Paragraph 6 in the div.</p>
</div>
<p>Paragraph 7. After a div.</p>
<p>Paragraph 8. After a div.</p>
</body>
</html>
```

Adjacent Sibling Selector

The + selector is used to select an element that is directly after another specific element.

The following example selects the first p element that are placed immediately after div elements:

Paragraph 1 in the div.

Paragraph 2 in the div.

Paragraph 3. After a div.

Paragraph 4. After a div.

Paragraph 5 in the div.

Paragraph 6 in the div.

Paragraph 7. After a div.

Paragraph 8. After a div.

General sibling selector

- The general sibling selector selects all elements that are next siblings of a specified element.
- The combinator for this relationship is the symbol ~
- Example: To select all <p> elements that are next siblings of <div> elements we write:

```
div ~ p {  
    background-color: yellow;  
}
```

- See an example in the next slide

General sibling selector

```
<!DOCTYPE html>
<html>
<head>
<style>
div ~ p {
  background-color: yellow;
}
</style>
</head>
<body>

<h2>General Sibling Selector</h2>

<p>The general sibling selector (~) selects all elements
that are next siblings of a specified element.</p>

<p>Paragraph 1.</p>

<div>
  <p>Paragraph 2.</p>
</div>

<p>Paragraph 3.</p>
<code>Some code.</code>
<p>Paragraph 4.</p>

</body>
</html>
```

General Sibling Selector

The general sibling selector (~) selects all elements that are next siblings of a specified element.

Paragraph 1.

Paragraph 2.

Paragraph 3.

Some code.

Paragraph 4.

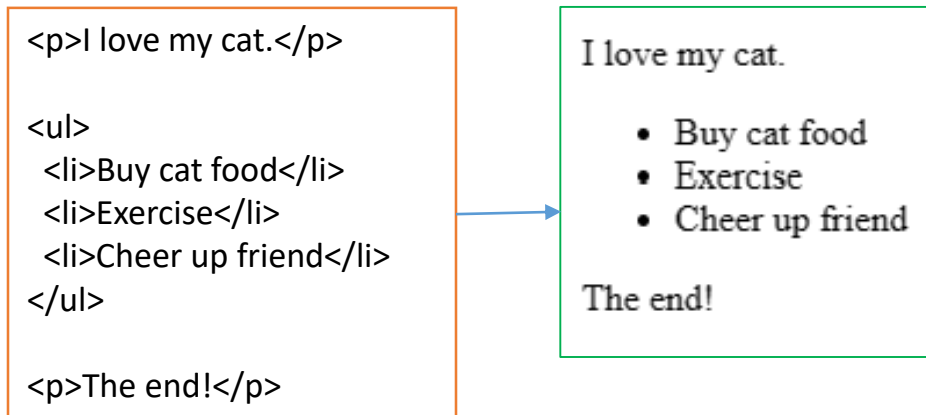
CSS Layouts

HTML Layout Techniques

- CSS page layout techniques allow us to take elements contained in a web page and control where they're positioned.
- Useful for creating multicolumn layouts.
- There are four different techniques to create multicolumn layouts.
 - Normal flow
 - CSS float property
 - CSS flexbox
 - CSS grid
 - CSS framework

Normal flow

- Normal flow is how the browser lays out HTML pages by default when you do nothing to control page layout.
- Example:



- By default, the browser will display the content in exact order in which it appears in the source code

Normal flow

- Normal flow displays content of an element with their default display flow.
 - Thus, block-level elements and inline elements will be displayed according to their default display
-
- By default, the browser will display the content in exact order in which it appears in the source code

Block-level element

- A block-level element is an elements that always starts on a new line
- Examples of block-level elements include:

```
<address>    <article>    <aside>    <blockquote>
<fieldset>   <figcaption> <figure>   <footer>
<main>       <nav>        <noscript> <ol>
<ul>         <video>
<canvas>     <dd>         <div>      <dl>      <dt>
<form>       <h1>-<h6>    <header>   <hr>      <li>
<p>          <pre>      <section>  <table>   <tfoot>
```

- Two commonly used block elements are: <p> and <div>.

Inline elements

- An inline element is the elements that does not start on a new line.
- Examples of inline elements include:

<code><a></code>	<code><abbr></code>	<code><acronym></code>	<code></code>	<code><bdo></code>	<code><big></code>	<code>
</code>	<code><button></code>	<code><cite></code>
<code><code></code>	<code><dfn></code>	<code></code>	<code><i></code>	<code></code>	<code><input></code>	<code><kbd></code>	<code><label></code>	<code><map></code>
<code><object></code>	<code><output></code>	<code><q></code>	<code><samp></code>	<code><script></code>	<code><select></code>	<code><small></code>	<code></code>	<code></code>
<code><sub></code>	<code><sup></code>	<code><textarea></code>	<code><time></code>	<code><tt></code>	<code><var></code>			

Display

- The default display of both block-level element and inline elements can be changed using **display property**.
- The display property specifies the display behavior (the type of rendering box) of an element.
- Syntax for display property is

display: value;

Display

- The default display of both block-level element and inline elements can be changed using **display property**.
- The display property specifies the display behavior (the type of rendering box) of an element.
- Syntax for display property is

display: value;

Display

- Some common values for display property include:

- **inline**

- **block**

- **flex**

- **grid**

Display

- **Inline:** Displays an element as an inline element

- **Syntax**

```
selector{  
    display:inline;}
```

- **Example:**

```
div > h1{  
    display:inline;}
```

Example: inline

```
<!DOCTYPE html>
<html>
<head>
<style>
body > p {
  display: inline;
}

</style>
</head>
<body>

<p>These two paragraphs generates inline boxes, and it
results in</p>

<p>no distance between the two elements.</p>

</body>
</html>
```

These two paragraphs generates inline boxes, and it results in no distance between the two elements.

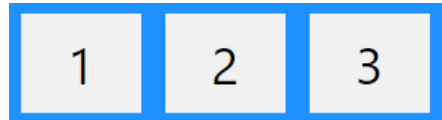
FlexBox

- CSS Flexbox, short for Flexible Box Layout Module, is a powerful layout tool in CSS that allows you to manipulate the positioning and alignment of elements in a container.
- It offers greater flexibility and control compared to traditional methods like floats and tables, making it a popular choice for modern web design.
- Flexbox is a one-dimensional layout method for arranging items in rows or columns.
- Items flex (expand) to fill additional space or shrink to fit into smaller spaces.
- Syntax:

```
section {  
    display: flex;  
}
```

Flex

- To start using the Flexbox model, you need to first define a flex container.
- Then define the items inside the created container
- The element in this image represents a flex container (the blue area) with three flex items.



Flex Container

- The flex container becomes flexible by setting the display property to flex

```
.flex-container {  
  display: flex;  
}
```

- See an example in next slide

```

<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  margin: 10px;
  padding: 20px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>Create a Flex Container</h1>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

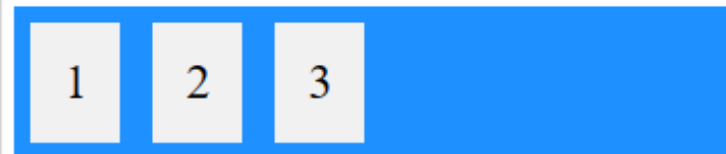
<p>A Flexible Layout must have a parent element
with the <em>display</em> property set to
<em>flex</em>.</p>

<p>Direct child elements(s) of the flexible
container automatically becomes flexible items.</p>

</body>
</html>

```

Create a Flex Container



A Flexible Layout must have a parent element with the *display* property set to *flex*.

Direct child element(s) of the flexible container automatically becomes flexible items.

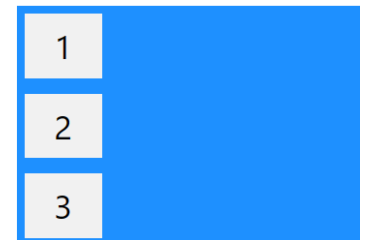
Flex

- The flex container properties are:
 - **flex-direction**
 - **flex-wrap**
 - flex-flow
 - justify-content
 - align-items
 - align-content

The flex-direction Property

- The flex-direction property defines in which direction the container wants to stack the flex items.
- The column value stacks the flex items vertically (from top to bottom):
- Example:

```
.flex-container {  
  display: flex;  
  flex-direction: column;  
}
```



- The flex elements will be displayed as shown in this image
- See the complete codes for this flex-direction property

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  flex-direction: column;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The flex-direction Property</h1>

<p>The "flex-direction: column;" stacks the flex
items vertically (from top to bottom):</p>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

</body>
</html>
```

The flex-direction Property

The "flex-direction: column;" stacks the flex items vertically (from top to bottom):



The flex-direction Property

- The column-reverse value stacks the flex items vertically (but from bottom to top):
- Example:

```
.flex-container {  
  display: flex;  
  flex-direction: column-  
reverse;  
}
```



- The flex elements will be displayed as shown in this image
- See the complete codes for this flex-direction property

```

<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  flex-direction: column-reverse;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The flex-direction Property</h1>

<p>The "flex-direction: column-reverse;" stacks the
flex items vertically (but from bottom to top):</p>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

</body>
</html>

```

The flex-direction Property

The "flex-direction: column-reverse;" stacks the flex items vertically (but from bottom to top):



Other values for flex-direction

- The row: The row value stacks the flex items horizontally (from left to right)
- The row-reverse: The row-reverse value stacks the flex items horizontally (but from right to left):

The flex-wrap Property

- The flex-wrap property specifies whether the flex items should wrap or not.
- Values for flex-wrap:
 - no-wrap: The wrap value specifies that the flex items will wrap if necessary
 - wrap: The nowrap value specifies that the flex items will not wrap (this is default):
 - The wrap-reverse value specifies that the flexible items will wrap if necessary, in reverse order

```
.flex-container {  
  display: flex;  
  flex-wrap: nowrap;  
}
```

```
.flex-container {  
  display: flex;  
  flex-wrap: wrap;  
}
```

```
.flex-container {  
  display: flex;  
  flex-wrap: wrap-reverse;  
}
```

Flex-wrap:nowrap

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  flex-wrap: nowrap;
  background-color: DodgerBlue;
}

.flex-container>div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The flex-wrap Property</h1>

<p>The "flex-wrap: nowrap;" specifies that the flex items
will not wrap (this is default):</p>

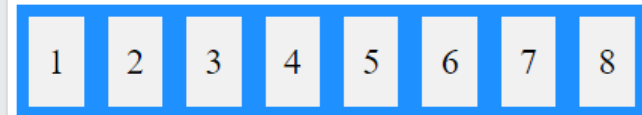
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
</div>

<p>Try resizing the browser window.</p>

</body>
</html>
```

The flex-wrap Property

The "flex-wrap: nowrap;" specifies that the flex items will not wrap (this is default):



Try resizing the browser window.

Flex-wrap:wrap

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  flex-wrap: wrap;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The flex-wrap Property</h1>

<p>The "flex-wrap: wrap;" specifies that the flex items will
wrap if necessary:</p>

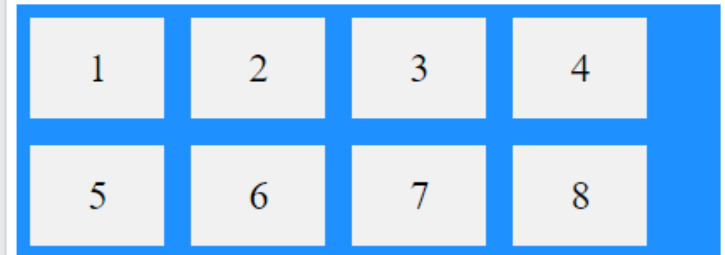
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
</div>

<p>Try resizing the browser window.</p>

</body>
</html>
```

The flex-wrap Property

The "flex-wrap: wrap;" specifies that the flex items will wrap if necessary:



Try resizing the browser window.

Flex-wrap:wrap-reverse

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  flex-wrap: wrap-reverse;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The flex-wrap Property</h1>

<p>The "flex-wrap: wrap-reverse;" specifies that the flex
items will wrap if necessary, in reverse order:</p>

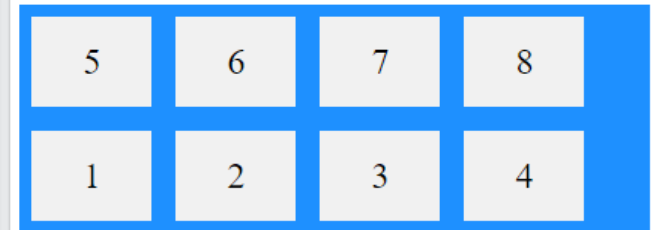
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
</div>

<p>Try resizing the browser window.</p>

</body>
</html>
```

The flex-wrap Property

The "flex-wrap: wrap-reverse;" specifies that the flex items will wrap if necessary, in reverse order:



Try resizing the browser window.

CSS Multiple Columns Layout

- If you have a really long list of elements, such as a list of all of the countries of the world, it can result in a lot of scrolling and time wasted for a user.
- It can also create excess whitespace on the page.
- With **CSS multicolumn property**, you can split this into multiple columns to help with both of these issues.

CSS Multi-column properties

- Multicolumn layout includes the following properties:
 - column-count
 - column-gap
 - column-rule-style
 - column-rule-width
 - column-rule-color
 - column-rule
 - column-span
 - column-width

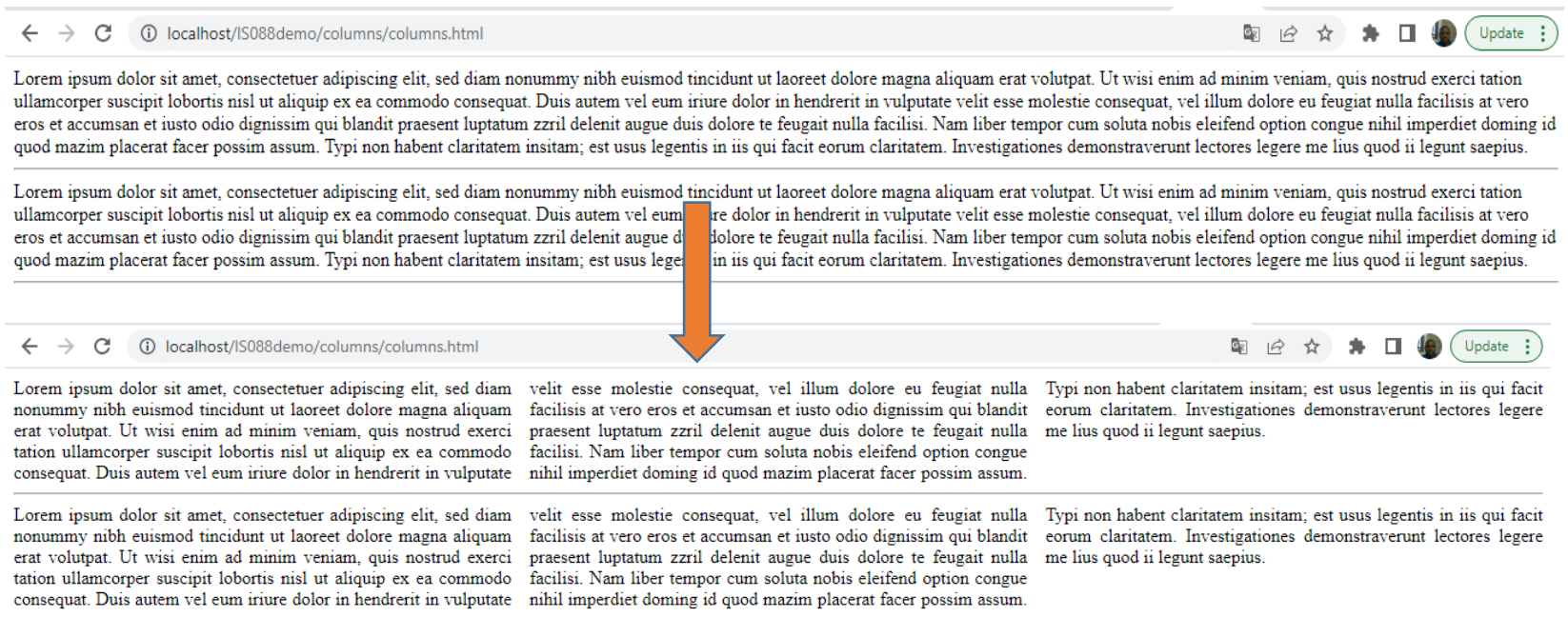
CSS Create Multiple Columns

- The column-count property specifies the number of columns an element should be divided into.
- The following css example will divide the text in the <div> element into 3 columns:

```
div {  
  column-count: 3;  
}
```

CSS column-count Property

- The **column-count** property in CSS is used to divide a portion of content inside any HTML element into a given number of columns.



column:count

```
<!DOCTYPE html>
<html>
<head>
<style>
.newspaper {
  column-count: 3;
}
</style>
</head>
<body>

<h1>Create Multiple Columns</h1>

<div class="newspaper">
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed
diam nonummy nibh euismod tincidunt ut laoreet dolore magna
aliquam erat volutpat. Ut wisi enim ad minim veniam, quis
nostrud exerci tation ullamcorper suscipit lobortis nisl ut
aliquip ex ea commodo consequat. Duis autem vel eum iriure
dolor in hendrerit in vulputate velit esse molestie consequat,
vel illum dolore eu feugiat nulla facilisis at vero eros et
accumsan et iusto odio dignissim qui blandit praesent luptatum
zzril delenit augue duis dolore te feugait nulla facilisi. Nam
liber tempor cum soluta nobis eleifend option congue nihil
imperdiet doming id quod mazim placerat facer possim assum.
</div>

</body>
</html>
```

Create Multiple Columns

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit	lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui	blandit praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.
--	--	--

CSS Specify the Gap Between Columns

- The column-gap property specifies the gap between the columns.
- The following css example specifies a 40 pixels gap between the columns:

```
div {  
    column-gap: 40px;  
}
```

column-gap

```
<!DOCTYPE html>
<html>
<head>
<style>
.newspaper {
  column-count: 3;
  column-gap: 40px;
}
</style>
</head>
<body>

<h1>Specify the Gap Between Columns</h1>

<div class="newspaper">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed
  diam nonummy nibh euismod tincidunt ut laoreet dolore magna
  aliquam erat volutpat. Ut wisi enim ad minim veniam, quis
  nostrud exerci tation ullamcorper suscipit lobortis nisl ut
  aliquip ex ea commodo consequat. Duis autem vel eum iriure
  dolor in hendrerit in vulputate velit esse molestie consequat,
  vel illum dolore eu feugiat nulla facilisis at vero eros et
  accumsan et iusto odio dignissim qui blandit praesent luptatum
  zzril delenit augue duis dolore te feugait nulla facilisi. Nam
  liber tempor cum soluta nobis eleifend option congue nihil
  imperdiet doming id quod mazim placerat facer possim assum.
</div>

</body>
</html>
```

Specify the Gap Between Columns

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit

lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit

praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.

CSS Column Rules

- The column-rule-style property specifies the style of the rule between columns:

```
div {  
  column-rule-style: solid;  
}
```

CSS Column Rules

```
<!DOCTYPE html>
<html>
<head>
<style>
.newspaper {
  column-count: 3;
  column-gap: 40px;
  column-rule-style: dashed;
}
</style>
</head>
<body>

<h1>Add a Rule Between the Columns</h1>

<div class="newspaper">
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed
diam nonummy nibh euismod tincidunt ut laoreet dolore magna
aliquam erat volutpat. Ut wisi enim ad minim veniam, quis
nostrud exerci tation ullamcorper suscipit lobortis nisl ut
aliquip ex ea commodo consequat. Duis autem vel eum iriure
dolor in hendrerit in vulputate velit esse molestie consequat,
vel illum dolore eu feugiat nulla facilisis at vero eros et
accumsan et iusto odio dignissim qui blandit praesent luptatum
zzril delenit augue duis dolore te feugait nulla facilisi. Nam
liber tempor cum soluta nobis eleifend option congue nihil
imperdiet doming id quod mazim placerat facer possim assum.
</div>

</body>
</html>
```

Add a Rule Between the Columns

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit	lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit	praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.
---	---	---

column-rule-color

- The column-rule-color property specifies the color of the rule between columns:

```
div {  
  column-rule-color: lightblue;  
}
```

column-rule-color

```
<!DOCTYPE html>
<html>
<head>
<style>
.newspaper {
  column-count: 3;
  column-gap: 40px;
  column-rule-style: solid;
  column-rule-width: 1px;
  column-rule-color: lightblue;
}
</style>
</head>
<body>

<h1>Set the Rule Color</h1>

<div class="newspaper">
  Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed
  diam nonummy nibh euismod tincidunt ut laoreet dolore magna
  aliquam erat volutpat. Ut wisi enim ad minim veniam, quis
  nostrud exerci tation ullamcorper suscipit lobortis nisl ut
  aliquip ex ea commodo consequat. Duis autem vel eum iriure
  dolor in hendrerit in vulputate velit esse molestie consequat,
  vel illum dolore eu feugiat nulla facilisis at vero eros et
  accumsan et iusto odio dignissim qui blandit praesent luptatum
  zzril delenit augue duis dolore te feugait nulla facilisi. Nam
  liber tempor cum soluta nobis eleifend option congue nihil
  imperdiet doming id quod mazim placerat facer possim assum.
</div>

</body>
</html>
```

Set the Rule Color

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit	lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit	praesent luptatum zzril delenit augue duis dolore te feugait nulla facilisi. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.
---	---	---

Sample Question

- Sample Question
- i. Create a webpage section using div element
- ii. Modify the created section by adding a class called newspaper
- iii. Write a css rule which divide the text in the div section into 4 column
- iv. Modify the css rule in (iii) by adding comment that specifies the purpose the rule
- v. Modify the css rule in (iv) by adding a property that specifies the color of the rule between columns to be lightblue
- vi. Modify the css rule in (v) by adding a property that specifies the style of the rule between columns to be dotted
- vii. Modify the css rule in (vi) by adding a property that specifies the style of the rule between columns to be dotted
- viii. Modify the css rule in (vii) by adding a property that specifies the gap between the columns to be 40px.
- ix. Under what circumstances is the css multicolumn property beneficial for a web design?